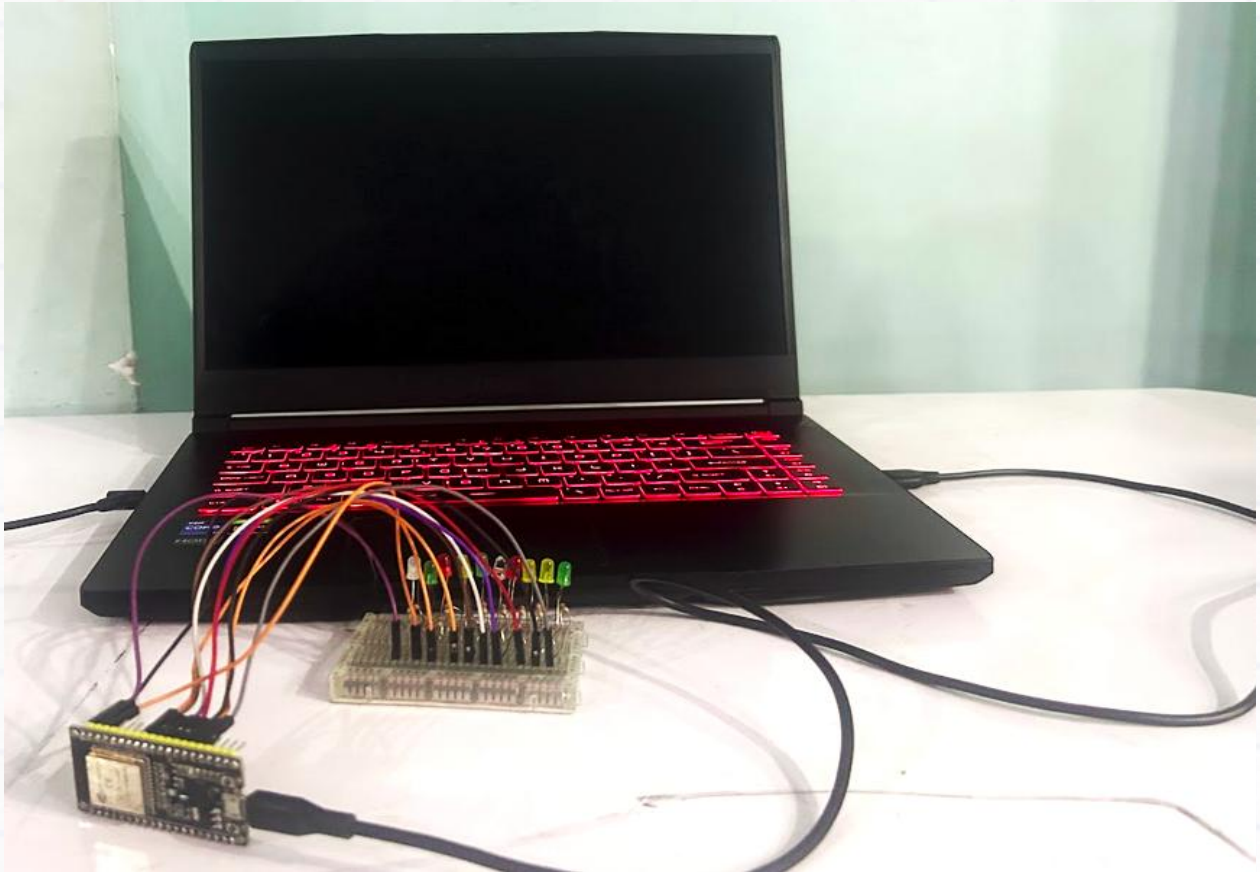


# Dancing LEDs



## Contents

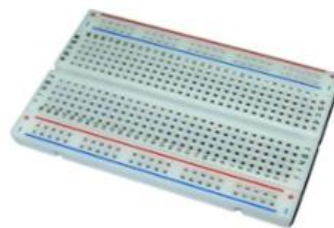
1. Outcome
2. Components
3. Concept
4. Hardware setup
5. Software
6. Tasks

## 1.Outcome

Get your build momentum started with this simple project to control LEDs with ESP32 C-6 module, Arduino IDE and create different visual effects.

## 2.Components

| Sno | Components                    | Quantity |
|-----|-------------------------------|----------|
| 1.  | Mini LED's                    | 10       |
| 2.  | 220-ohm Resistor              | 10       |
| 3.  | Breadboard                    | 1        |
| 4.  | Jumper wires (Male to Female) | 11       |
| 5.  | ESP32 C6 Dev Module           | 1        |
| 6.  | USB to Type – C cable         | 1        |



BreadBoard



LED



Resistor



Jumper wires



USB

## 3. Concept

The power to LEDs are supplied from the GPIO pins of the ESP32 C-6 board through a resistor. The timing of the power supply to the LEDs are controlled through the program and visual effects are created by switching ON & OFF.



## 4. Hardware setup

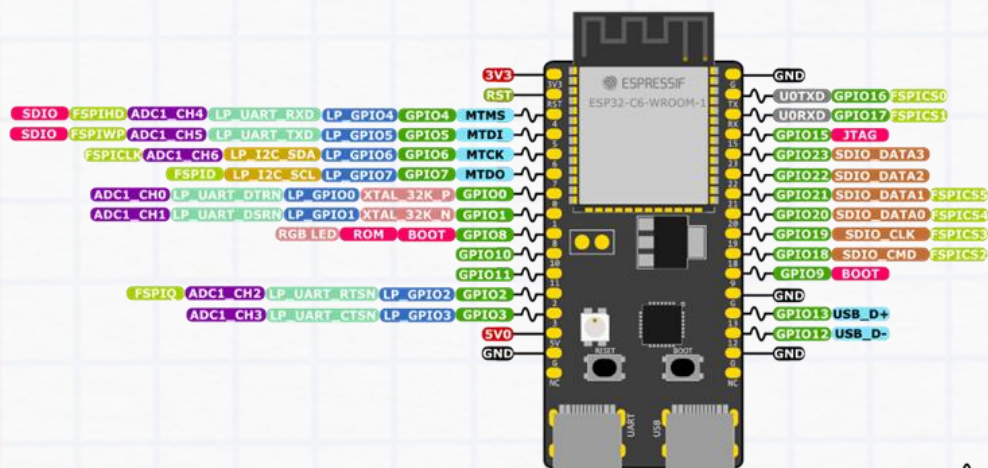
**Safety Tip:** Always ensure that the connections to the components are correct and completed before connecting the power supply to the ESP32 C-6 board.



In LEDs, the positive electrode is the longer wire, and the negative electrode is the shorter wire. In a typical application, a resistor is connected in series with the LED to limit the amount of current and ensure the LED doesn't get damaged.

### The ESP32 C-6 Module Pinout Diagram

ESP32-C6-DevKitC-1



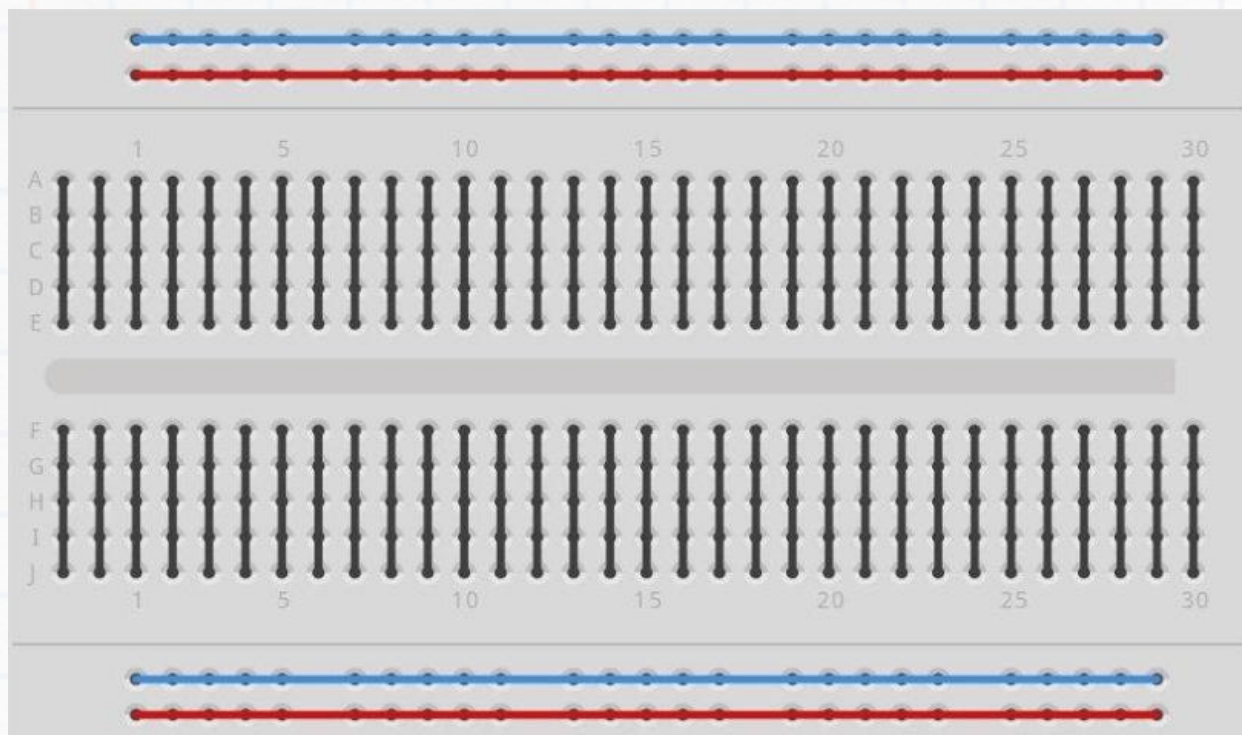
**ESP32-C6 Specs**  
 32-bit RISC-V single-core @160MHz  
 Wi-Fi IEEE 802.11 ax 2.4GHz + Bluetooth LE 5  
 + IEEE 802.15.4 (Zigbee and Thread)  
 512 KB SRAM (21 KB for cache)  
 320 KB ROM  
 30 or 22 GPIOs, 3x SPI, 2x UART, 1x I2C, RMT  
 LED PWM 6ch, 1x 12-bit ADC with 7ch, TWAI®  
 USB Serial/JTAG, ETM, MCPWM, SDIO Slave

**Source:** [Datasheet](#)

## 4.a Breadboard Connection Rails

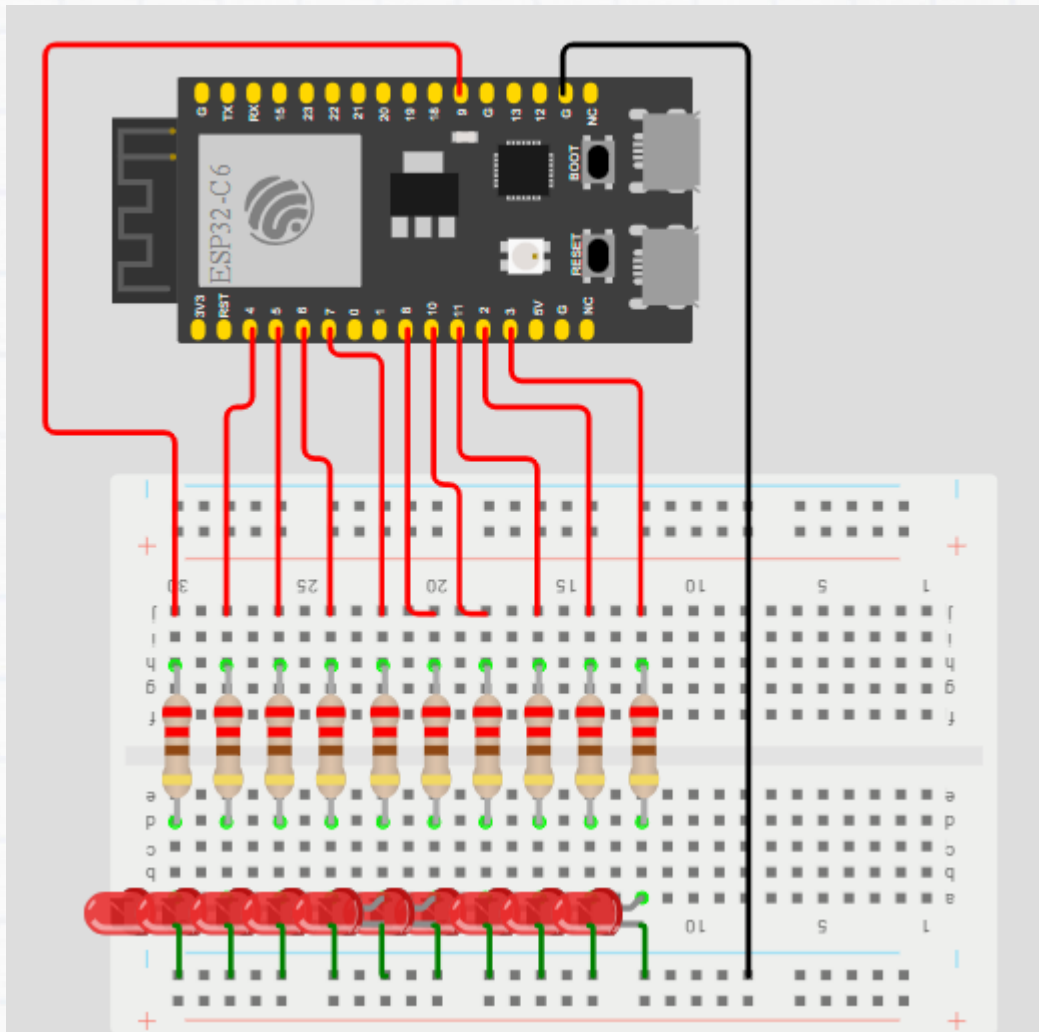
Blue – Ground Rail

Red – Power Rail



**Source :** [learn.sparkfun.com](http://learn.sparkfun.com)

## 4.b Component Connections



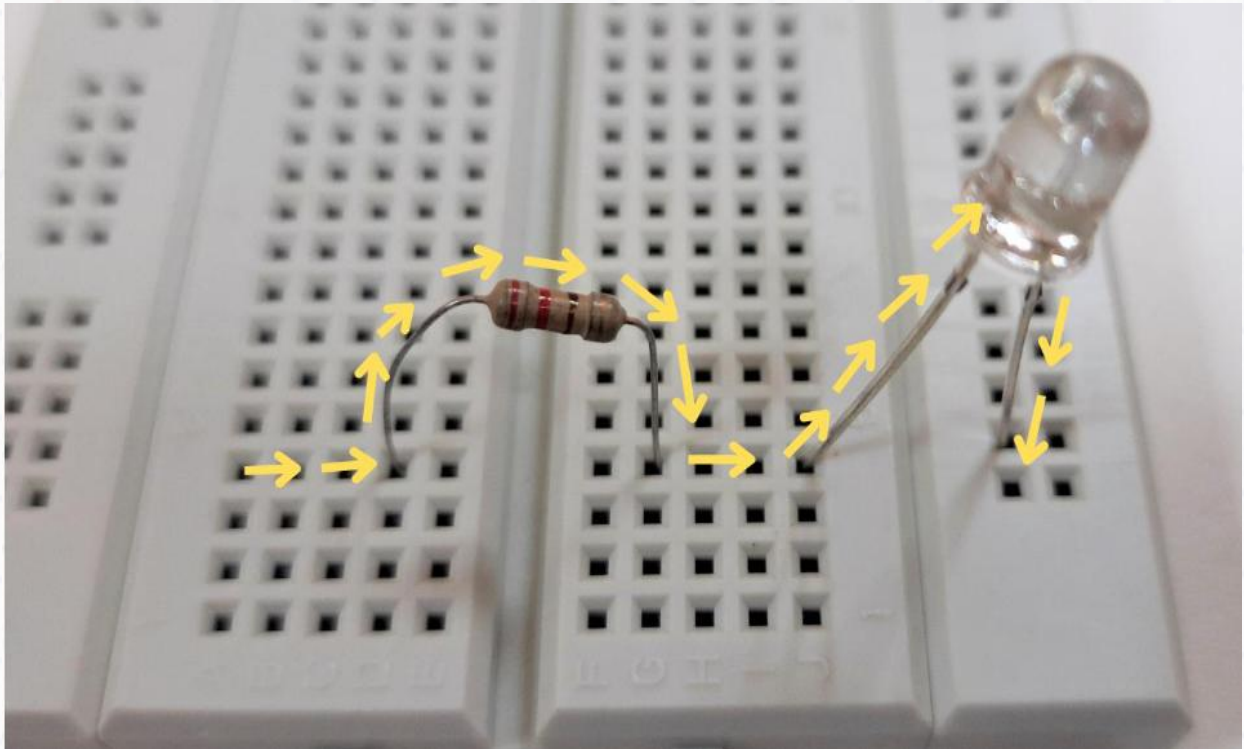
**Circuit Simulation done on:** <https://wokwi.com/esp32>

Follow the detailed steps in the following pages to complete the circuit and play with the LED Controls.



## Detailed Circuit Connection Steps

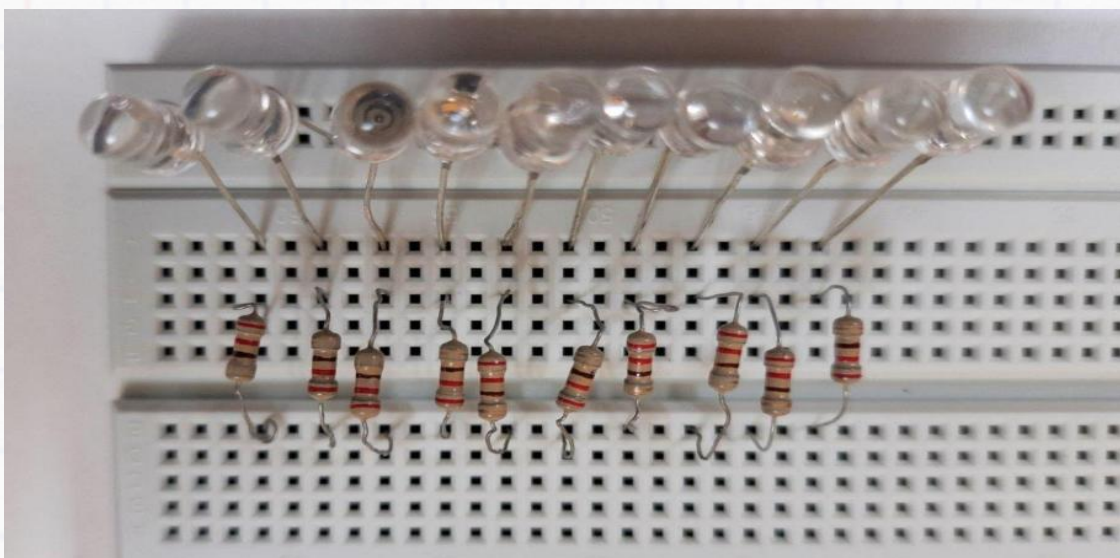
### Step 1



Connect the longer leg of the LED to the resistor and the shorter leg to the ground terminal. The circuit connection is as shown by the yellow arrows in the above image.

### Step 2

Repeat the connections similarly for the remaining LEDs (10 in total) as shown above. Ensure that the connections for each LED setup are in the same horizontal rows.



### Step 3

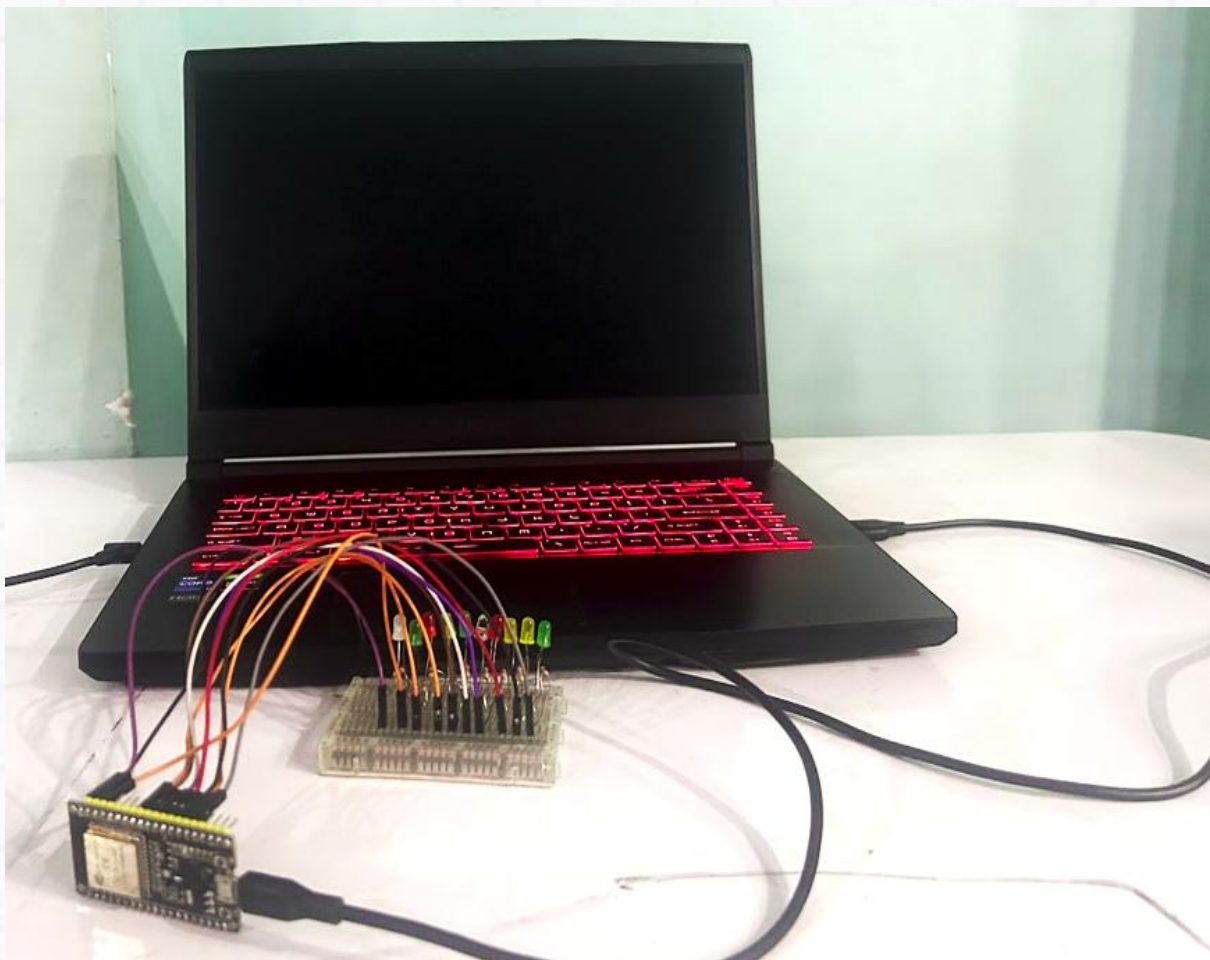
The shorter terminals of the LEDs in the breadboard are connected to the Ground terminal of the ESP32-C6 development board using a jumper wire.

### Step 4

The longer terminals of the LEDs with the resistors are connected to the GPIO pins of the ESP32C-6 using jumper wires.

GPIO pin numbers are mentioned in the software code of the following pages.

### Step 5



The complete circuit of LEDs and ESP32 C-6.

Now, Let's move on to the Software part of the project.



## 5. Software

1. The code demonstrates how to control multiple LEDs connected to an Arduino board. The LEDs will blink in a sequence, each turning on and off for one second.
- These constants define the GPIO pins to which the LEDs are connected.

```
const int LED1 = 3;  
const int LED2 = 4;  
const int LED3 = 5;  
const int LED4 = 6;  
const int LED5 = 7;  
const int LED6 = 8;  
const int LED7 = 9;  
const int LED8 = 10;  
const int LED9 = 18;  
const int LED10 = 19;
```

- The `setup()` function runs once when the microcontroller is powered on or reset. Here, it sets each LED pin as an output.

```
void setup() {  
  // Initialize digital pins as outputs  
  pinMode(LED1, OUTPUT);  
  pinMode(LED2, OUTPUT);  
  // write the code for remaining 8 LEDs in similar format.  
}
```

- The `loop()` function runs repeatedly after the `setup()` function. Initially, it contains code to turn on LED1, wait for one second, turn it off, and wait for another second.

```
void loop() {  
  digitalWrite(LED1, HIGH); // turn LED1 on  
  delay(1000);              // wait for 1 second  
  digitalWrite(LED1, LOW);  // turn LED1 off  
  delay(1000);              // wait for 1 second  
}
```

The sample code in the above loop() function shows how to blink LED1. To extend the functionality to control LEDs 2 through 10. For each LED, add similar code as shown for LED1, modifying the pin numbers accordingly.

Ensure each LED blinks in sequence, with each turning on for one second and off for one second before the next LED in the sequence blinks.

- Create a new sketch with name ***danceLED.ino***
- Copy and Paste the below code into your sketch and add the remaining LEDs code into the loop() function.

```
const int LED1 = 3;
const int LED2 = 4;
const int LED3 = 5;
const int LED4 = 6;
const int LED5 = 7;
const int LED6 = 8;
const int LED7 = 9;
const int LED8 = 10;
const int LED9 = 18;
const int LED10 = 19;

void setup()
{
  // Initialize digital pins as outputs
  pinMode(LED1, OUTPUT);
  pinMode(LED2, OUTPUT);

  // Write the remaining code for other LEDs in the same format.
}

void loop()
{
  digitalWrite(LED1, HIGH); // turn LED1 on
  delay(1000);               // wait for 1 second
  digitalWrite(LED1, LOW);  // turn LED1 off
  delay(1000);               // wait for 1 second

  //Write the remaining code for other LEDs in similar format.
}
```

## 2. Save the Sketch:

- Click on **File > Save As....**
- Name the file "**dancing\_led**" (without quotes).
- Make sure the file extension is **.ino** (this is automatically handled by Arduino IDE).
- Choose a location on your computer where you want to save the file.
- Click Save.

### General Instructions – To Upload Code to ESP32 C-6

## 1. Connect Arduino Board:

Connect your Arduino board to your computer using a USB cable.

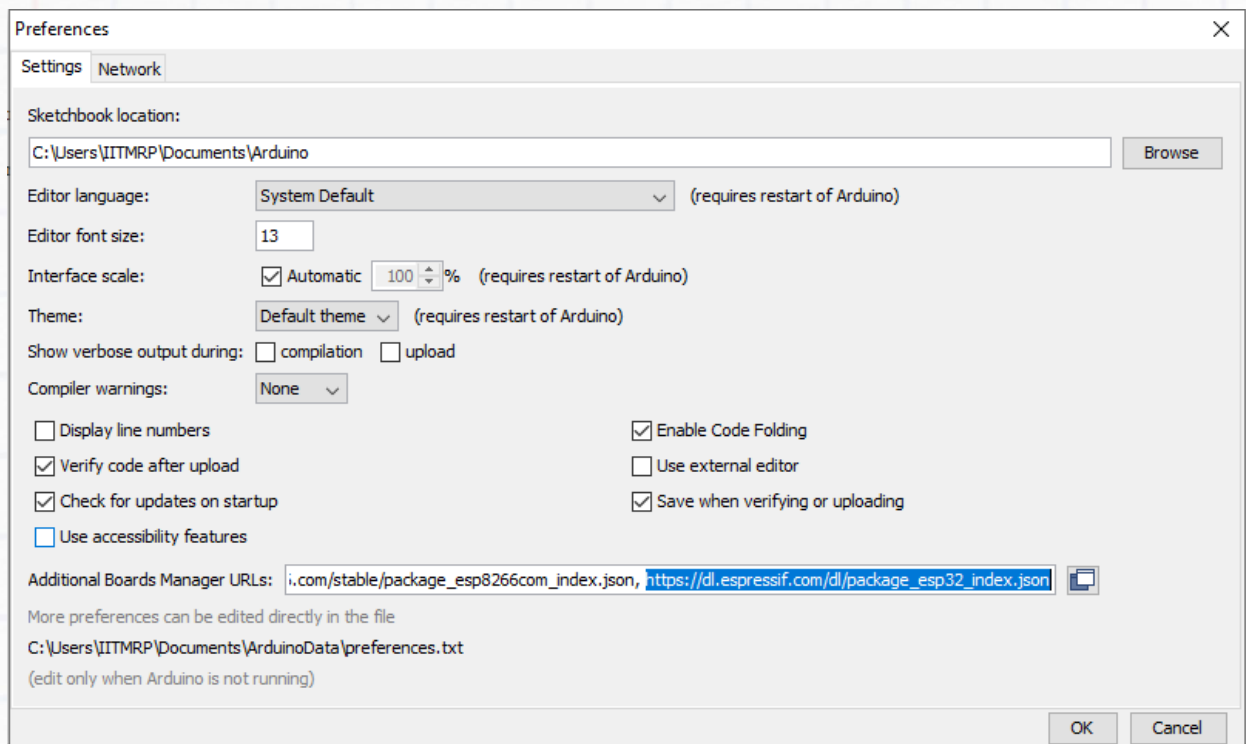
## 2. Select Board and Port:

- Open the Arduino IDE if it's not already open.
- Go to **File > Preferences**. In '**Additional Boards Manager URLs**', if the textbox is empty, paste the following link:



[https://dl.espressif.com/dl/package\\_esp32\\_index.json](https://dl.espressif.com/dl/package_esp32_index.json)

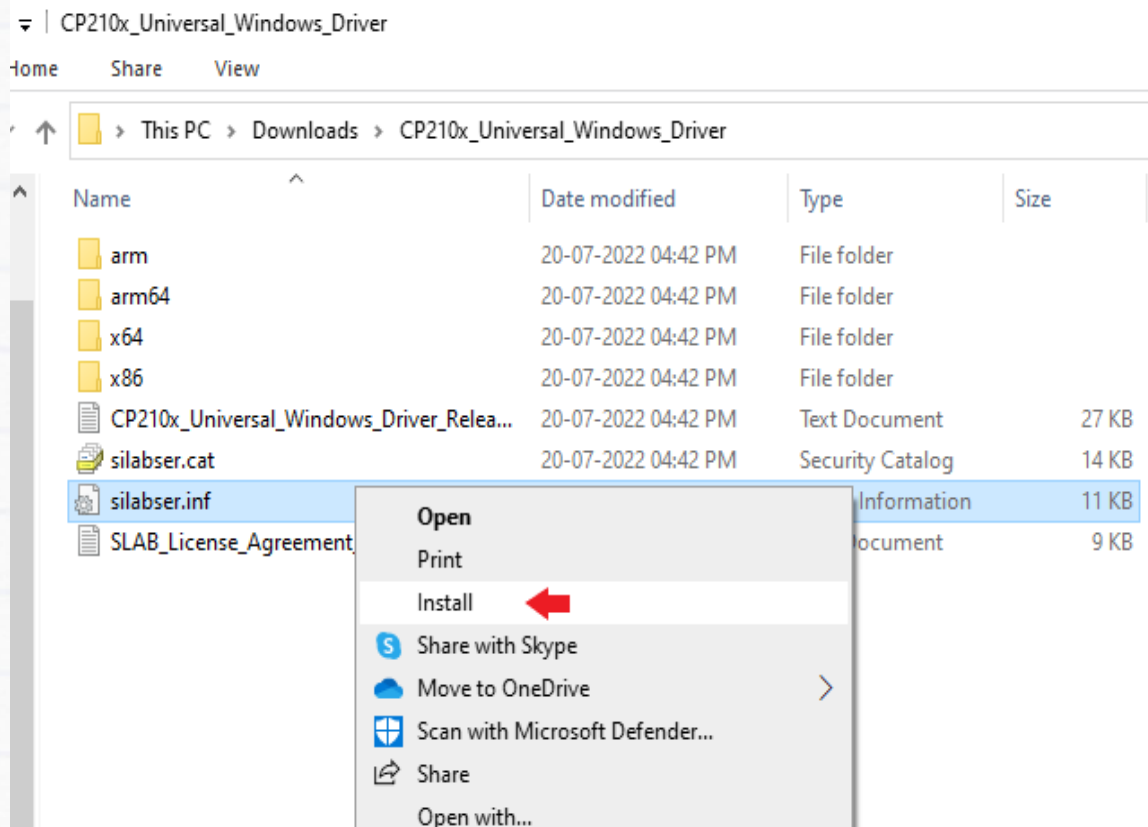
If the textbox already has some other links, put a comma after them and paste the above link.



- Next, go to **Tools > Board > Boards Manager**. In the search bar on the top, search 'esp32' (version 2.0.11) and install the package. This will take some time to install.

### 3. Select Port:

- From this [link](#), download the **CP210x Driver** for your OS (eg. CP210x Universal Windows Driver). Extract the ZIP folder. Right-click the **silabser.inf** file inside and select install. Follow the prompts until installation is successful. Restart your PC once installation completes.



- Go to **Tools > Port** and select the port to which your Arduino board is connected. The port will typically show as **COMx** on Windows, **/dev/cu.usbmodemxxxx** or **/dev/tty.usbmodemxxxx** on macOS, or **/dev/ttyUSBx** or **/dev/ttyACMx** on Linux.

#### 4. Verify and Compile Sketch:

- Click on the verify button (checkmark icon) or go to **Sketch > Verify/Compile** to compile your sketch. This step ensures that there are no syntax errors in your code.

#### 5. Upload Sketch:

- Once the sketch compiles successfully, click on the upload button (right arrow icon) or go to **Sketch > Upload**. This action will compile the sketch again (if necessary) and upload it to your Arduino board.

#### 6. Monitor Upload Progress:

- The status bar at the bottom of the Arduino IDE will show the progress of the upload process. During this time, you might see the onboard LED on your Arduino board blink rapidly, indicating that the sketch is being uploaded.

## 7. Upload Completion:

- Once the upload is complete, the status bar will display "**Done uploading**".
- If there are any errors during the upload process, carefully read the error messages in the output window at the bottom of the Arduino IDE. Common issues include incorrect board selection, missing drivers, or incorrect port selection.

## 8. Verify Operation:

After uploading, your Arduino board should start executing the "dancing LED" sequence as per the code you wrote.

# 6.Tasks

## Workshop Tasks

1. Make the blinking pattern for shorter time intervals (100ms)
2. Create a Wave Pattern with the blinking LEDs that goes from left to right and right to left. Make LEDs dance! - Program a new creative pattern with the LEDs. Take a short video clip of it and share it with the Build Club Discord community!

## Post Workshop Tasks

1. Execute blinking LED wave pattern using timer function to control every individual LED.
2. Execute two different types of wave pattern at same instantaneous time.